

Next-Day Hourly Stockout Prediction for Fresh Retail Supply Chains

*A report submitted in fulfillment for the award of the degree of
Bachelors of Technology*

in

Information Technology

Bachelors of Technology Project

Final Evaluation Report

By

Anurag Thakur

2023IMG-008

Samyak Choudhary

2023IMG-044

Vibhor Kumar

2023IMG-054

Under the Supervision of

Dr. Sunil Kumar

Department of Information Technology



विश्वजीवनामृतं ज्ञानम्

ABV-INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
AND MANAGEMENT GWALIOR
GWALIOR, INDIA

DECLARATION

We hereby certify that the work, which is being presented in the report, entitled **Next-Day Hourly Stockout Prediction for Fresh Retail Supply Chains**, in fulfillment of the requirement for the award of the degree of **Bachelors of Technology (B.Tech)** and submitted to the institution is an authentic record of my/our own work carried out during the period under the supervision of **Dr. Sunil Kumar**. We also cited the reference about the text(s)/figure(s)/table(s) from where they have been taken.

Dated:

Signature of the candidates

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Dated:

Signature of supervisor

Acknowledgements

We would like to express our deepest gratitude to our supervisor, Dr. Sunil Kumar, for his invaluable guidance and unwavering support throughout this project. It was his relentless motivation that inspired us to think beyond the immediate limitations of our initial approaches. He constantly challenged us to explore more possibilities, design extensive new experiments, and delve deeper into advanced research. Dr. Kumar was the defining reason we tested so many architectural variations and ultimately succeeded in finding the perfect, highly efficient final model. His mentorship has been instrumental in shaping both our theoretical understanding and our practical engineering skills.

Anurag, Samyak, and Vibhor

Abstract

In this project, our team tackled the difficult problem of hourly stockouts in fast grocery delivery. Traditional forecasting usually looks at daily totals, which does not work well for quick commerce where missing an item for even a few hours means losing customers. Using the FreshRetailNet-50K dataset, we focused on predicting stockouts hour-by-hour for specific stores. We started by testing basic models like Vanilla RNNs and LSTMs to understand how they handle time sequences. After testing different setups, we built our own model called the Dual-Stream Inventory Shortcut LSTM. This custom model processes demand and inventory data separately and sends the latest inventory numbers directly to the final output, which improved accuracy a lot. Finally, we compared our models against the latest linear methods using a strict testing framework with Focal Loss. We found that a model called the Multi-Kernel DLinear worked the best. It captures both quick panic-buying and longer trends very well while being simple and fast (using only 13K parameters). It achieved the highest Hour-Level F1-Score of 0.8131, making it a great solution for real-world store predictions.

Contents

List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Introduction	1
1.2 Motivation	2
2 Literature Survey	3
2.1 Traditional Inventory Forecasting	3
2.2 The Deep Learning Shift in Time-Series	3
2.2.1 Recurrent Neural Networks (RNNs) and LSTMs	4
2.2.2 Transformers and Linear Alternatives	4
2.2.3 Retail Aggregation Limitations	4
3 Thesis Objectives and Deliverables	5
3.1 Problem Formulation	6
3.2 Dataset Overview	6
4 System Architecture and Methodology	7
4.1 Baseline Analysis	7
4.1.1 Baseline SimpleRNN	7
4.1.2 Baseline Vanilla and Canonical LSTM	8
4.1.3 Baseline Transformers	8
4.2 Figuring Out Feature Importance	9
4.3 Building Our Custom LSTMs	10
4.3.1 Our Final Custom LSTM: Dual-Stream Inventory Shortcut	10
4.4 DLinear (Decomposition Linear) Architectures	11
4.4.1 Standard DLinear	11
4.4.2 Multi-Kernel DLinear	11
5 Tasks Completed	13

6	Results and Conclusion	15
6.1	Standardized Evaluation Framework	15
6.2	Final Empirical Results	16
6.3	Conclusion	17

List of Figures

4.1	The Canonical LSTM processing pipeline for 24-hour stockout prediction. .	8
4.2	Integrated Gradients Heatmap revealing the disproportionate importance of recent inventory signals compared to historical static features.	9
4.3	The Dual-Stream Inventory Shortcut LSTM Architecture. Notice the dashed shortcut preserving the $t = 14$ state.	10
4.4	The Standard DLinear Architecture. Trend and residual mapped via independent linear layers.	11
4.5	The Multi-Kernel DLinear extraction phase. Multiple kernels capture volatility at different frequencies.	12
6.1	Final Empirical Comparison: The linear architectures systematically outperform the recurrent LSTM bounds.	16
6.2	Broad comparison across model families, illustrating the parameter-efficiency dominance of DLinear variants on the FreshRetailNet-50K task.	17

List of Tables

4.1	Systematic Progression of Recurrent Architectural Variations	10
6.1	Performance Comparison of Final Evaluated Architectures	16

1

Introduction

1.1 Introduction

The way we shop has changed a lot recently. People are moving away from traditional physical stores and switching to ultra-fast delivery services. These fast grocery delivery services (often called "quick commerce") promise to get items to your door in ten to thirty minutes. To make this happen, companies need a large network of small, local warehouses, often known as "dark stores."

In this fast-paced setup, managing inventory is no longer just about saving money; it is the biggest challenge to keeping the business running and customers happy. Traditional retail forecasting systems usually look at sales on a daily or weekly basis. If a regular supermarket runs out of milk at 6:00 PM but restocks it by 7:00 AM the next day, the system only sees that a lot of milk was sold that day. It completely misses the fact that the shelf was empty for hours.

However, in quick commerce, if an item is out of stock for four hours during the evening rush, customers cannot finish their orders. They will likely abandon their carts and switch to a competitor immediately. Because of this, a computer model that just predicts how much will be sold tomorrow is not good enough. What we really need is a detailed, hour-by-hour forecasting tool that predicts exactly when a specific shelf will run empty.

Our team explored how to build and improve sequential deep learning models specifically to solve this high-frequency, store-level stockout problem. Using the FreshRetailNet-

1. Introduction

50K dataset, we moved away from traditional daily supply research and focused on making highly specific, hourly predictions.

1.2 Motivation

The main motivation for our project comes from the huge financial and operational problems in fresh grocery delivery. Fresh produce and perishable goods go bad very quickly. If store managers stock too much to avoid running out, a lot of food spoils and gets wasted. On the other hand, if they stock too little to prevent spoilage, they risk running out of stock during the day, which means losing customers forever in the quick commerce world.

Early in our research, our team tried using traditional inventory forecasting methods. While these classic models laid the groundwork, they did not fit the reality of modern dark stores. They treated time as a simple, unchanging variable and assumed sales happened evenly throughout the day.

When we tested a basic Tabular Transformer model on the FreshRetailNet-50K dataset (using 96 hidden dimensions, 2 transformer layers, and 4 attention heads), it only achieved an accuracy of 55.83% and an AUC of 0.5257—which is barely better than guessing randomly. This poor performance clearly showed our team that predicting stockouts is not a simple classification problem. It is a complex sequence problem that relies on the timeline of events, shopping cycles, and everyday operational noise.

Because of this, we were motivated to drop static models and use dynamic time-series architectures instead. We looked into everything from Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs) to Decomposition Linear (DLinear) models, hoping to capture exactly how and when inventory runs out.

2

Literature Survey

Forecasting retail inventory and analyzing time-series data has changed a lot over the past decade. This chapter covers the different approaches researchers have taken to solve these problems.

2.1 Traditional Inventory Forecasting

In the past, predicting inventory relied on standard statistical models, mainly Autoregressive Integrated Moving Average (ARIMA) and Exponential Smoothing (ETS). While these models are good at math, they usually only look at one variable at a time. Because of this, they struggle to include outside factors like promotional discounts, weather changes, or holidays, which are very important in retail.

Also, it is very slow and hard to run these traditional models for hundreds of thousands of different products across many stores. In modern retail, a basic statistical model often fails because it cannot learn how different things affect each other (for example, it cannot realize that a discount on Product A in one city might have a similar effect as a discount on Product B in another city).

2.2 The Deep Learning Shift in Time-Series

To fix the problems with those older statistical models, researchers started using deep learning for forecasting.

2.2.1 Recurrent Neural Networks (RNNs) and LSTMs

Recurrent Neural Networks, especially Long Short-Term Memory (LSTM) networks created by Hochreiter and Schmidhuber [1], completely changed how we process data over time. Older RNNs used to forget earlier data in a sequence (a problem known as the vanishing gradient), but LSTMs use an internal memory and special "gates" (Input, Forget, and Output gates) to remember things better. This helps them capture trends over longer periods, making them the standard choice in the industry for forecasting over time.

2.2.2 Transformers and Linear Alternatives

More recently, researchers have adapted Transformer models for time-series forecasting. Transformers were originally built to process language all at once using something called "self-attention," leading to new models like the Informer [2] and Autoformer [3]. Some have also tried using them on standard tabular data [4].

However, an important paper by Zeng et al. (2023) [5] showed that simpler Decomposition Linear (DLinear) models could actually beat complex Transformers on many time-series tests, especially when looking at short periods of history. This discovery strongly influenced the later stages of our team's research.

2.2.3 Retail Aggregation Limitations

Traditional models often just predict a single, daily total for demand [6]. This hides exactly when an item goes out of stock during the day. Because of this limitation, our team knew we had to focus on detailed, hourly predictions instead.

3

Thesis Objectives and Deliverables

To fix the problems with older forecasting models and successfully predict when items will run out of stock during the day, our team set the following clear goals for this project:

- (i) **Set up the Forecasting Problem:** We needed to clearly define the stockout problem mathematically. We wanted our model to look at the last 14 days of history (like sales and discounts) and predict an exact 24-hour breakdown of whether a specific store will have enough stock tomorrow.
- (ii) **Test Basic Recurrent Models:** We planned to train basic Vanilla RNN and Vanilla LSTM models first. This helped us see where they struggle and what goes wrong when they try to process messy, real-world retail data.
- (iii) **Build Our Own Custom Model:** We aimed to build a specialized neural network. Our final design, the Dual-Stream Inventory Shortcut LSTM, processes the unpredictable demand data separately from the hard inventory facts, and it makes sure the latest inventory updates are not forgotten.
- (iv) **Compare Against the Latest Linear Models:** We wanted to test our custom LSTMs against newer, lightweight linear models (like DLinear and Multi-Kernel DLinear) using a very strict testing process to see which approach was actually better.
- (v) **Deliver a Working Solution:** Ultimately, our main goal was to pick one fast, easy-to-understand model (which ended up being the Multi-Kernel DLinear) that

gives the most accurate hourly predictions, so it could actually be used in real, fast-paced fulfillment centers.

3.1 Problem Formulation

To set this up, we take historical retail data $X \in \mathbb{R}^{T \times D}$, where $T = 14$ days is the time window we look back on, and D is the number of different features we track. Our goal is to train a function $f : X \rightarrow Y$.

The target output Y is a 24-dimensional vector that tells us exactly if an item is in stock for every hour of the next day ($T + 1$):

$$Y = [y_1, y_2, \dots, y_{24}]^T \quad \text{where} \quad y_i \in \{0, 1\} \quad (3.1)$$

In this setup, if $y_i = 1$, it means the item is out of stock during hour i , and if $y_i = 0$, it means there is plenty of inventory.

3.2 Dataset Overview

To hit these goals, our team ran all our experiments using the **FreshRetailNet-50K** dataset. Our mentor provided this data, and it gives a very realistic picture of how modern retail works.

It includes daily records for specific combinations of cities, stores, and products. Importantly, the dataset mixes direct sales numbers with outside factors that affect how people shop:

- **Sales and Inventory Facts:** Basic numbers like total sales, past stockouts, and daily activity.
- **Promotions:** Information on discounts or store events that make people buy things faster than usual.
- **Outside Events:** National holidays and weather conditions that change overall shopping behavior.

By feeding all this information in order, our models learned the difference between normal items selling out over time and items selling out suddenly because of a sale. This dataset paved the way for all the tests we talk about in the next chapters.

4

System Architecture and Methodology

This chapter breaks down the different models our team built to tackle the FreshRetailNet-50K hourly stockout problem. We followed a step-by-step process: we started by testing basic models to see how they behave, then we used machine learning explainability to figure out which features actually mattered, built our own custom LSTM model, and finally tested out some of the newest linear models.

4.1 Baseline Analysis

To get a solid starting point, we set up a standard test where the models looked at 14 days of history to predict the next 24 hours. We wanted to see how well these basic models could handle messy retail data before we tried to invent our own solution.

4.1.1 Baseline SimpleRNN

We started with the most basic model for time-series data: a Simple Recurrent Neural Network (RNN). We set it up with a hidden size of 64, giving it about 14,744 parameters to learn. While it was fast to run, it had a lot of trouble remembering things over the full 14 days (a common issue called the vanishing gradient problem). It only achieved an Hour-Level F1-Score of 0.7750, which told our team that simple RNNs are not powerful enough to understand complex shopping habits.

4. System Architecture and Methodology

4.1.2 Baseline Vanilla and Canonical LSTM

To fix the problems we saw with the SimpleRNN, we built a Baseline LSTM. LSTMs use special "gates" (Forget, Input, and Output) that help the model hold onto important information from earlier in the 14-day window. Mathematically, the LSTM updates its hidden state h_t and cell state c_t at each step t like this:

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (\text{Forget Gate}) \quad (4.1)$$

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (\text{Input Gate}) \quad (4.2)$$

$$\tilde{c}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c) \quad (\text{Candidate Cell State}) \quad (4.3)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{Final Cell State}) \quad (4.4)$$

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (\text{Output Gate}) \quad (4.5)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{Hidden State}) \quad (4.6)$$

At first, we fed all 13 original features into it and set a hidden size of 96. This Vanilla LSTM was pretty heavy, with about 44,952 parameters, but it performed better with an F1-Score of 0.7852.

After figuring out that we did not need all 13 features (which we explain in Section 4.2), we shrank this down into what we call the **Canonical LSTM**. We fed it just the 6 most important features and lowered the hidden size to 32.

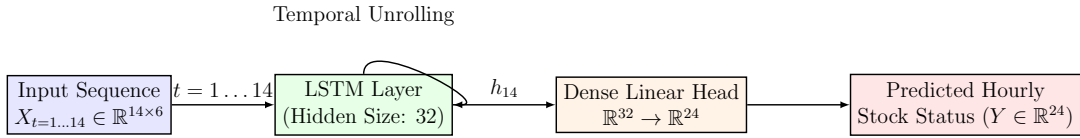


Figure 4.1: The Canonical LSTM processing pipeline for 24-hour stockout prediction.

This Canonical LSTM was much faster (only 19,992 parameters) and slightly more accurate (0.7880 F1-Score). However, it still had a flaw: it forced the LSTM gates to track crazy demand spikes and actual physical inventory limits inside the exact same hidden state h_t , which confused the model.

4.1.3 Baseline Transformers

Because everyone uses Transformers nowadays, our team also tested some heavy Transformer models, specifically PatchTST [7] and Temporal Fusion Transformers (TFT) [8]. Even though they were massive (over 280,000 parameters), they actually did worse than our basic LSTM, getting F1-Scores around 0.7540 to 0.7685. Transformers are built to

look at everything at once, which didn't fit well with our short 14-day history where physical stock goes down step-by-step. They were just too big for the job.

4.2 Figuring Out Feature Importance

Early on, our team wondered if we really needed all 13 features the dataset provided. Sometimes giving a model too much messy data just confuses it (which is probably why the Transformers failed). To fix this scientifically, we used a technique called Integrated Gradients (IG) [9] to show us exactly what the model was paying attention to over the 14 days.

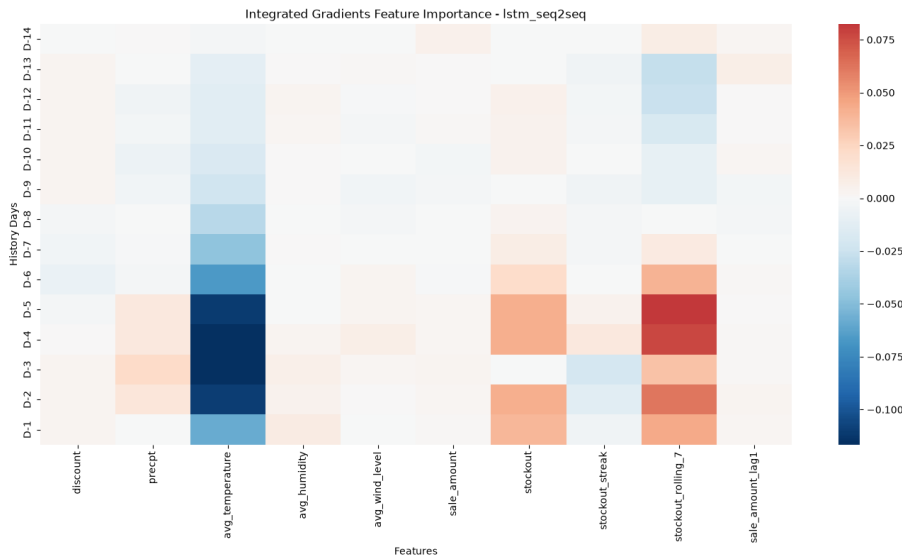


Figure 4.2: Integrated Gradients Heatmap revealing the disproportionate importance of recent inventory signals compared to historical static features.

As shown in Figure 4.2, this test showed us two really important things that changed how we built the rest of our models:

- (i) **Recent Data is King:** The data from the last two days ($t = 13$ and $t = 14$) was way more important than what happened two weeks ago.
- (ii) **Inventory Beats Demand:** Hard numbers about how much stock is left (like `stock_hour6_22_cnt`) were far more useful than messy data about past sales or discounts.

Based on this, our team threw out the bad data and kept only 6 essential features, giving our models a much cleaner signal to learn from.

4.3 Building Our Custom LSTMs

Armed with just the best features, our team started building a custom model piece-by-piece. We tested different structural setups to see what actually improved predictions. Table 4.1 shows the steps we took.

Table 4.1: Systematic Progression of Recurrent Architectural Variations

Stage	Architecture	Parameters	F1-Score
1	Baseline SimpleRNN (h=64)	14,744	0.7750
2	Dual-Stream SimpleRNN	5,016	0.7782
3	Baseline Vanilla LSTM (h=96)	44,952	0.7852
4	Feature-Reduced Canonical LSTM (h=32)	19,992	0.7880
5	Dual-Stream Inventory Shortcut LSTM	4,600	0.7941

4.3.1 Our Final Custom LSTM: Dual-Stream Inventory Shortcut

To fix the confusion happening inside the Canonical LSTM, we built our final custom model: the **Dual-Stream Inventory Shortcut LSTM**.

We split the 6 good features into two groups: 4 features for Demand, and 2 features for Inventory. We gave each group its own separate LSTM to process. On top of that, because our heatmap showed that the very last day’s inventory is the most important thing, we added a special "Shortcut." This shortcut takes the inventory numbers from Day 14 and sends them straight to the final output, completely bypassing the LSTMs. This guarantees that the LSTM never accidentally "forgets" the most important data.

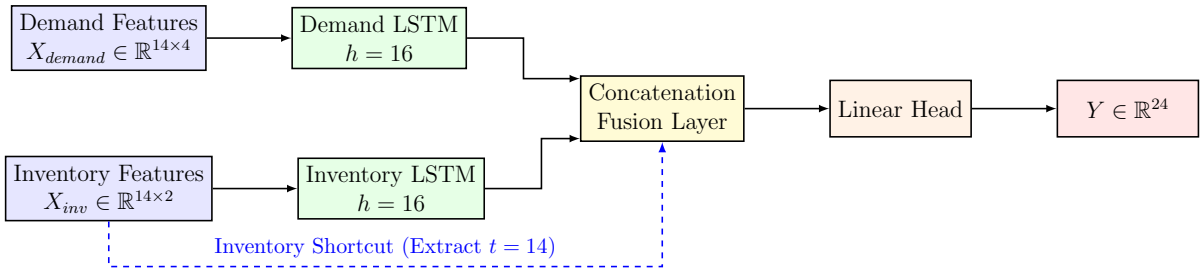


Figure 4.3: The Dual-Stream Inventory Shortcut LSTM Architecture. Notice the dashed shortcut preserving the $t = 14$ state.

This new design dropped the parameter count all the way down to just 4,600, but pushed our F1-Score up to 0.7941, making it our best recurrent model by far.

4.4 DLinear (Decomposition Linear) Architectures

After doing everything we could with LSTMs, our team wanted to see if newer Decomposition Linear (DLinear) models could do better. According to Zeng et al. (2023) [5], these simple, single-layer models that split time-series data into Trends and Seasons can sometimes beat complex deep learning models because they are simpler and don't get stuck trying to remember long sequences.

4.4.1 Standard DLinear

Standard DLinear takes the input data X and splits it into a Trend piece (X_{trend} , found by averaging recent data) and a Seasonal piece ($X_{seasonal}$, found by subtracting the Trend from the original). The math looks like this:

$$X_{trend} = \text{AvgPool1D}(X) \quad (4.7)$$

$$X_{seasonal} = X - X_{trend} \quad (4.8)$$

Then, it just uses standard linear layers ($W_{trend}, W_{seasonal}$) to map these pieces directly to the 24-hour prediction. It is very fast and easy to understand:

$$Y_{pred} = (W_{trend} \cdot X_{trend}) + (W_{seasonal} \cdot X_{seasonal}) \quad (4.9)$$

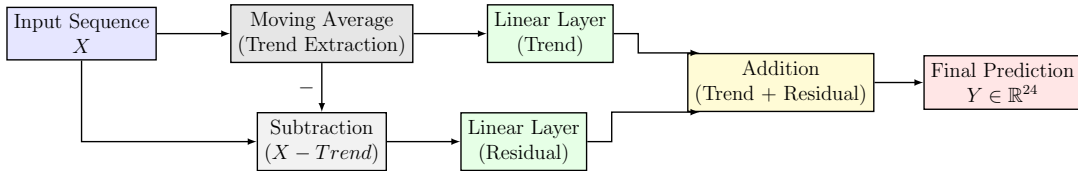


Figure 4.4: The Standard DLinear Architecture. Trend and residual mapped via independent linear layers.

4.4.2 Multi-Kernel DLinear

To make DLinear work even better for the crazy ups and downs of grocery shopping, our team built the **Multi-Kernel DLinear**. Instead of just doing one average to find the trend, this model runs multiple averages at the same time (like $k = 3, 5, 7$) to catch both sudden panic-buying and smooth weekly habits. It groups all these trends together like this:

4. System Architecture and Methodology

$$X_{trend} = \text{Concat}(\text{AvgPool1D}_{k=3}(X), \text{AvgPool1D}_{k=5}(X), \text{AvgPool1D}_{k=7}(X)) \quad (4.10)$$

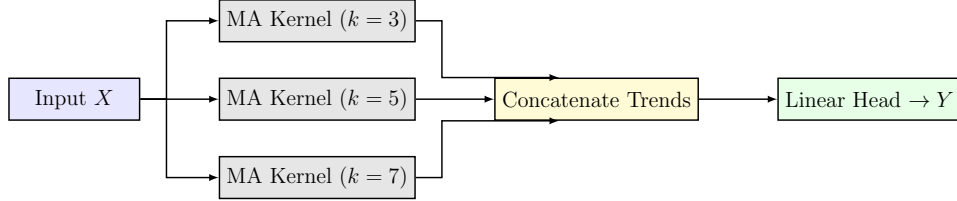


Figure 4.5: The Multi-Kernel DLinear extraction phase. Multiple kernels capture volatility at different frequencies.

5

Tasks Completed

To find the best model for this project, our team followed a strict three-month plan. Here is a breakdown of what we accomplished each month:

(i) Month 1: Research and Setting Up the Basics

- We started by researching older forecasting methods to understand how inventory prediction used to be done.
- We cleaned up the FreshRetailNet-50K dataset and organized it so our models could read it.
- We built and trained basic Vanilla RNN and Vanilla LSTM models to see how they performed, which gave us a baseline to beat (44K parameters, 0.785 F1-Score).

(ii) Month 2: Feature Engineering and Building Our Custom Model

- We used Integrated Gradients to figure out which features actually mattered, and threw away the bad ones, leaving us with just 6 strong features.
- We designed a new "Dual-Stream" setup that processes crazy demand spikes separately from hard inventory facts.
- We finished building our custom *Dual-Stream Inventory Shortcut LSTM*, which shrank the model down to just 4,600 parameters while actually beating the bigger baseline models.

5. Tasks Completed

(iii) Month 3: Testing Linear Models and Final Results

- We set up a very strict testing process (using Focal Loss and dynamic threshold scanning) to make sure our results were completely fair and accurate.
- We built and tested the *Multi-Kernel DLinear* model to see if it could handle both sudden panic-buying and longer weekly trends.
- We collected all our final numbers, created the graphs, proved that the Multi-Kernel DLinear was our best model, and wrote this final thesis report.

6

Results and Conclusion

As our project went on, we knew we had to compare our custom LSTM models against the newest linear models to see which was truly better. This chapter explains how we tested everything fairly, shows our final numbers, and gives our overall conclusion.

6.1 Standardized Evaluation Framework

To make sure we were comparing apples to apples, our team set up a very strict testing process. This stopped any model from looking better just because of lucky guesses or messy data:

- **Dataset Scope:** We only tested the top 15 busiest store-product combinations. This stopped the models from learning useless patterns from items that never sell anyway.
- **Holdout Strategy:** We used a strict timeline split. We hid the last 15 days of the dataset from the models completely, and only used those days for the final test.
- **Optimization:** In retail, stockouts don't happen as often as regular sales. To fix this imbalance, we trained the models using **Focal Loss** instead of standard Binary Cross-Entropy. The Focal Loss is defined mathematically as:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t) \tag{6.1}$$

6. Results and Conclusion

where p_t is the model’s estimated probability for the stockout class. We configured the focusing parameter $\gamma = 2.0$ and the balancing factor $\alpha = 0.5$.

- **Dynamic Thresholding:** Instead of just picking a standard 50/50 cutoff for predictions, we had the computer scan all cutoffs from 10% to 90% on a practice set to find the perfect spot that maximized the F1-Score.

6.2 Final Empirical Results

We tested our main recurrent models against a few versions of the DLinear architecture (which we talked about in Chapter 4). You can see the final scores and how big each model is in Table 6.1.

Table 6.1: Performance Comparison of Final Evaluated Architectures

Architecture	Parameters	Hour-Level F1
Vanilla LSTM	44,952	0.7850
Dual-Stream Inventory Shortcut LSTM	4,600	0.7941
Standard DLinear	6,768	0.7932
Hourly Slot-Specific DLinear	6,744	0.8058
Multi-Kernel DLinear	13,539	0.8131

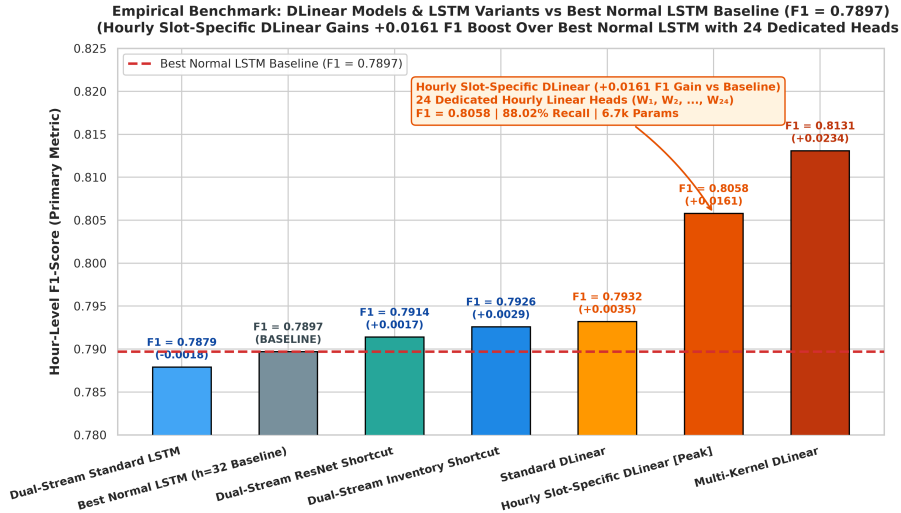


Figure 6.1: Final Empirical Comparison: The linear architectures systematically outperform the recurrent LSTM bounds.

The numbers clearly showed that the **Multi-Kernel DLinear** was the overall winner. It hit a peak F1-Score of **0.8131** while using only 13,539 parameters. It beat both our custom LSTM and all the basic baseline models (as you can see in Figure 6.1).

When we looked at all the different types of models we tried (Figure 6.2), we saw that while the older recurrent models eventually stop improving, the linear models are faster, smaller, and just better at predicting exact hourly stockouts.

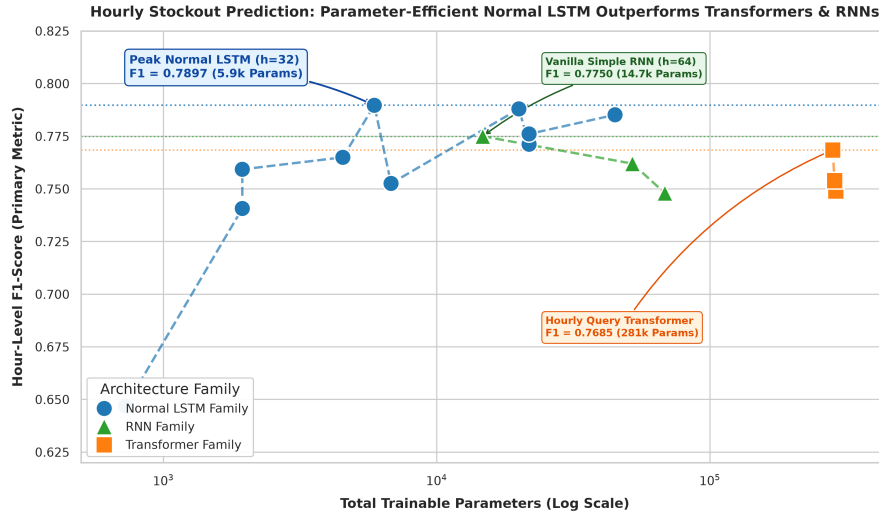


Figure 6.2: Broad comparison across model families, illustrating the parameter-efficiency dominance of DLinear variants on the FreshRetailNet-50K task.

6.3 Conclusion

This project took on the real-world problem of predicting hourly stockouts in fast-paced grocery delivery. By moving away from predicting daily totals and focusing on specific hour-by-hour predictions, our team tackled a problem that actually costs delivery apps a lot of customers.

Going through all these different models taught our team a few big lessons:

- (i) **Good Features Matter Most:** Getting rid of bad data and focusing on a few strong features helped the models way more than just making the models bigger.
- (ii) **Routing the Data Helps:** When building LSTMs, separating the demand numbers from the inventory numbers, and taking a "shortcut" for the most recent day's data, stopped the model from forgetting the important stuff.
- (iii) **Linear Models Win Here:** In our strict testing, we proved that simple Decomposition Linear (DLinear) models are actually better for this specific short-term forecasting job than complex deep learning LSTMs.

6. Results and Conclusion

While our Dual-Stream Inventory Shortcut LSTM was our best idea for a recurrent model, the **Multi-Kernel DLinear** is the final model we recommend using in the real world. Because it doesn't have messy recurrent memory, it is very small (only 13,539 parameters) and fast. By running multiple averages at the same time, it easily handles the short 14-day history, catching both sudden panic-buying and smooth weekly trends to get the highest F1-Score of 0.8131.

By predicting exactly which hour a shelf will empty, this tool can help grocery managers restock shelves before they run out. This means less spoiled food, less wasted money, and happier customers when they open the app to order.

References

- [1] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [2] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, “Informer: Beyond efficient transformer for long sequence time-series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11106–11115, 2021.
- [3] H. Wu, J. Xu, J. Wang, and M. Long, “Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting,” in *Advances in Neural Information Processing Systems*, vol. 34, pp. 22419–22430, 2021.
- [4] Y. Gorishniy, I. Rubachev, V. Khrulkov, and A. Babenko, “Revisiting deep learning models for tabular data,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 18932–18943, 2021.
- [5] A. Zeng, M. Chen, L. Zhang, and Q. Qi, “Are transformers effective for time series forecasting?,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 11121–11128, 2023.
- [6] A. A. Syntetos, M. Z. Babai, J. E. Boylan, and P. Syntetos, “Supply chain forecasting: Theory, practice, their intersection and the future,” *European Journal of Operational Research*, vol. 252, no. 1, pp. 1–54, 2016.
- [7] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, “A time series is worth 64 words: Long-term forecasting with transformers,” in *International Conference on Learning Representations*, 2023.
- [8] B. Lim, S. Ö. Arık, N. Loeff, and T. Pfister, “Temporal fusion transformers for interpretable multi-horizon time series forecasting,” *International Journal of Forecasting*, vol. 37, no. 4, pp. 1748–1764, 2021.
- [9] M. Sundararajan, A. Taly, and Q. Yan, “Axiomatic attribution for deep networks,” in *International conference on machine learning*, pp. 3319–3328, PMLR, 2017.

- [10] C. Challu, K. G. Olivares, B. N. Oreshkin, F. Garza, M. Mergenthaler-Canseco, and A. Dubrawski, “N-hits: Neural hierarchical interpolation for time series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, pp. 6989–6997, 2023.
- [11] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in neural information processing systems*, vol. 30, 2017.